

TECHNICAL DOCUMENTATION

Deliva — Delivery Booking Platform

A line-by-line explanation of the codebase, the technology stack, and how the Google-Maps-style address autocomplete was built — written for project defense.

Stack React Router 7 · Vite · Hono · Neon Postgres

Feature covered Pickup/delivery address autocomplete

Companion doc Defense Q&A

CONTENTS

- | | |
|--|-------------------------------------|
| 1. 1. What the project is | 7. 7. How the map search was built |
| 2. 2. The technology stack | 8. 8. Input validation |
| 3. 3. Folder structure | 9. 9. Environment variables |
| 4. 4. How routing works | 10. 10. What .ts / .tsx / .jsx mean |
| 5. 5. The booking page, line by line | 11. 11. Running the project locally |
| 6. 6. The autocomplete component, line by line | 12. 12. Limitations & next steps |

1. What the project is

Deliva is a last-mile delivery booking application, in the same product category as Bolt or Uber but for parcels instead of people. A customer fills in a sender, a receiver, and a package, the app estimates a cost, and a dispatch rider is assigned. There are three user roles reflected in the dashboard: **customer**, **rider**, and **admin**.

The specific feature this document focuses on is the **pickup and delivery address fields** on the booking form: as the user types, a dropdown of real address suggestions appears (like Bolt/Uber), and selecting one captures the exact address text plus its latitude/longitude coordinate — the "point" on the map.

2. The technology stack

Every tool below was already a dependency in `package.json` or was added for this feature. None of it is a framework the app "could" use — this is what actually runs.

LAYER	TOOL	WHAT IT'S DOING HERE
UI library	<code>react</code> 18	Renders the pages as components; manages state with hooks (<code>useState</code> , <code>useEffect</code>).
App framework	<code>react-router</code> 7 (framework mode)	File-based routing, data loading, and server-side rendering (SSR) scaffolding. A folder under <code>src/app</code> becomes a URL.
Build tool	<code>vite</code> 6	Compiles JSX/TypeScript, serves the app in development with hot-reload, bundles it for production.
Compiler	<code>esbuild</code> + <code>babel</code>	Vite uses esbuild for speed; a small Babel pass is layered on top for one plugin (<code>styled-jsx</code>) that esbuild can't do alone.
Server runtime	<code>hono</code> via <code>react-router-hono-server</code>	A lightweight web server (an alternative to Express) that serves the app's pages and its API routes from one process.
Styling	Tailwind CSS 3	Utility classes (<code>rounded-x1</code> , <code>bg-gray-50</code> , ...) instead of hand-written CSS files.
Icons	<code>lucide-react</code>	The <code>MapPin</code> , <code>User</code> , <code>Loader2</code> , etc. icons used throughout.
Animation	<code>motion</code> (the successor to Framer Motion)	The fade/scale-in on the booking success screen.
Authentication	<code>@auth/core</code> (Auth.js) + <code>argon2</code>	Email/password sign-in. Passwords are hashed with argon2, never stored in plain text.
Database	Neon (serverless Postgres) via <code>@neondatabase/serverless</code>	Stores users, sessions, and delivery records. No ORM — plain tagged-template SQL.

Location search	LocationIQ Autocomplete API (plain <code>fetch</code> , REST)	Turns typed text into address suggestions with coordinates. Detailed in §7.
Package manager	Bun (<code>bun.lock</code>) in the original project; this local copy runs on npm	Installs and locks dependency versions.
Language	JavaScript + TypeScript, mixed	Pages are <code>.jsx</code> ; framework plumbing is <code>.ts</code> / <code>.tsx</code> . See §10.

3. Folder structure

```
apps/web/
├─ src/
│  ├─ app/                # every page and API route lives here
│  │  ├─ booking/page.jsx # the delivery booking form (this doc's focus)
│  │  ├─ dashboard/       # customer / rider / admin dashboards
│  │  ├─ track/page.jsx   # public delivery tracking page
│  │  ├─ api/             # server-side route handlers (route.js)
│  │  │  └─ deliveries/route.js
│  │  ├─ root.tsx         # the HTML shell every page renders inside
│  │  └─ routes.ts        # walks this folder tree to build the route table
│  ├─ components/
│  │  └─ PlaceAutocompleteInput.jsx # the address-suggestion input
│  ├─ client-integrations/ # thin wrappers around third-party SDKs
│  ├─ utils/              # shared hooks (useUser, useUserProfile, sql client)
│  │  └─ auth.js          # Auth.js configuration (sign-in/sign-up logic)
├─ vite.config.ts         # build + dev-server configuration
├─ package.json
└─ .env                  # secrets and API keys (never committed)
```

Two folder names carry special meaning for React Router's file-based routing: a folder holding `page.jsx` becomes a visitable URL; a folder holding `route.js` under `api/` becomes a JSON endpoint. That mapping is explained next.

4. How routing works

There is no hand-written list of URLs. `src/app/routes.ts` recursively reads the `src/app` folder at build time and turns its shape into a route table:

- `src/app/booking/page.jsx` → `/booking`
- `src/app/track/page.jsx` → `/track`
- `src/app/api/deliveries/route.js` → `/api/deliveries` (a JSON endpoint, not a page)
- A folder named `[id]` becomes a URL parameter — `src/app/api/riders/jobs/[id]/route.js` → `/api/riders/jobs/:id`

This is why creating a new page is just "add a folder and a `page.jsx` file" rather than registering a route somewhere else.

5. The booking page, line by line

`src/app/booking/page.jsx` is a single component that renders a 3-step form (Sender → Receiver → Package) and posts the result to the API. Walking through it in order:

Imports and setup

```
"use client";
```

Tells the framework this component needs the browser (state, event handlers) rather than being renderable purely on the server. Anything with `useState` or `onClick` needs this.

```
import { useState } from "react";
import { Package, Truck, ArrowLeft, ... } from "lucide-react";
import { useUserProfile } from "@/utils/useUserProfile";
import { motion } from "motion/react";
import PlaceAutocompleteInput from "@components/PlaceAutocompleteInput";
```

`@/` is a path alias configured in `vite.config.ts` that always points at `src/`, so imports don't need long relative paths like `../../components/...`. `useUserProfile` fetches the logged-in user's saved name/phone to pre-fill the form.

```
const INPUT_CLS =
  "w-full bg-gray-50 border border-gray-100 rounded-xl py-4 px-5 outline-none focus:ring-2
```

One Tailwind class string reused on every input, so all fields look identical and a style change only happens in one place.

```
const NG_PHONE_REGEX = /^(0\d{10}|\+234\d{10})$/;
const NAME_REGEX = /^[A-Za-z'-]+(?:\s+[A-Za-z'--]+)+$/;

const isValidNgPhone = (value) => NG_PHONE_REGEX.test(value.trim());
const isValidFullName = (value) => NAME_REGEX.test(value.trim());
```

Two small validator functions, explained in full in §8. Declared outside the component so they aren't recreated on every re-render.

Component state

```
const { data: user } = useUserProfile();
const [step, setStep] = useState(1);
const [loading, setLoading] = useState(false);
const [success, setSuccess] = useState(null);
const [error, setError] = useState(null);
```

`step` drives which of the three form panels is visible. `loading` / `success` / `error` track the network request when the form is finally submitted.

```
const [formData, setFormData] = useState({
  sender_name: user?.name || "",
  sender_phone: user?.phone || "",
  pickup_address: "",
  pickup_lat: null,
  pickup_lng: null,
  ...
});
```

One object holds every field in the form — a common React pattern called a **controlled form**: the input's visible value always comes from state, never from the DOM directly.

`pickup_lat` / `pickup_lng` were added for this feature to remember the exact map coordinate behind the chosen address, even though the field only displays text.

Behaviour functions

```
const calculateCost = () => {
  const baseFare = 500;
  const distanceRate = 100 * 10;
  let weightRate = 200;
  if (formData.weight > 10) weightRate = 800;
  else if (formData.weight > 5) weightRate = 400;
  const priorityCharge = formData.priority === "Express" ? 1000 : 0;
  return baseFare + distanceRate + weightRate + priorityCharge;
};
```

A pure pricing calculation: flat base fare + a fixed distance charge (currently a hard-coded 10 km, not yet computed from the real pickup/delivery coordinates — see §12) + a weight tier + an express surcharge.

```
const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData((prev) => ({ ...prev, [name]: value }));
};
```

One generic handler for every plain `<input>` / `<select>`. It reads the field's `name` attribute and updates just that key in `formData`, leaving the rest untouched via the `...prev` spread.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setLoading(true);
  setError(null);
  try {
    const cost = calculateCost();
    const res = await fetch("/api/deliveries", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...formData, cost }),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || "Failed to create delivery request");
    setSuccess(data.delivery);
    ...
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};
```

```
}  
};
```

`e.preventDefault()` stops the browser's default full-page-reload form submission. The rest is a standard **fetch** → **JSON** → **error-or-success** flow: everything currently in `formData` (including the `lat/lng` fields) is serialized to JSON and POSTed to the `/api/deliveries` route handler (`$3's route.js`).

The address fields (this feature's core)

```
<PlaceAutocompleteInput  
  required  
  name="pickup_address"  
  value={formData.pickup_address}  
  onChange={(text) =>  
    setFormData((f) => ({ ...f, pickup_address: text, pickup_lat: null, pickup_lng: null }))  
  }  
  onSelect={({ address, lat, lng }) =>  
    setFormData((f) => ({ ...f, pickup_address: address, pickup_lat: lat, pickup_lng: lng })  
  }  
  className={INPUT_CLS}  
  placeholder="Start typing an address..."  
>
```

Two different callbacks matter here. `onChange` fires on every keystroke — it updates the visible text and *clears* the stored coordinate, because free-typed text with no selection has no confirmed point yet. `onSelect` fires only when the user clicks/enters a suggestion from the dropdown — it fills in the address *and* the real latitude/longitude returned by LocationIQ. This is the exact mechanism that produces "suggestions and points."

Step gating

```
disabled={
  !isValidFullName(formData.sender_name) ||
  !isValidNgPhone(formData.sender_phone) ||
  !formData.pickup_address
}
```

The "Next Step" button is disabled — not just when a field is empty, but when it's filled with something that fails validation. This is what actually enforces the Nigerian phone format and two-word name rule described in §8; without this, a malformed value could still slip through to the next step.

Success screen

Once `handleSubmit` succeeds, `success` holds the delivery record returned by the API (including its `tracking_id`), and the component's *early return* (`if (success) { return (...) }` near the top) swaps the entire form out for a confirmation card with links to the dashboard and the tracking page.

6. The autocomplete component, line by line

`src/components/PlaceAutocompleteInput.jsx` is a self-contained, reusable input — it has no idea it's being used for "pickup" vs "delivery"; the booking page tells it what to do via props (`value` , `onChange` , `onSelect`).

State it keeps track of

```
const [suggestions, setSuggestions] = useState([]);
const [open, setOpen] = useState(false);
const [loading, setLoading] = useState(false);
const [activeIndex, setActiveIndex] = useState(-1);
const containerRef = useRef(null);
const debounceRef = useRef(null);
const abortRef = useRef(null);
```

`suggestions` is the list currently shown in the dropdown. `activeIndex` is which row is highlighted for keyboard (arrow-key) navigation. The three `useRef` s hold mutable values that must survive re-renders *without* triggering a re-render themselves when changed — a timer id, an abort controller, and a reference to the outer `<div>` for click-outside detection.

Closing the dropdown on outside click

```
useEffect(() => {
  function handleClickOutside(e) {
    if (containerRef.current && !containerRef.current.contains(e.target)) {
      setOpen(false);
    }
  }
  document.addEventListener("mousedown", handleClickOutside);
  return () => document.removeEventListener("mousedown", handleClickOutside);
}, []);
```

Listens for clicks anywhere on the page; if the click target is outside this component's own container, the dropdown closes. The empty `[]` dependency array means this listener is attached once when the component mounts, and the returned function removes it when the component unmounts — a standard React cleanup pattern to avoid leaking listeners.

Fetching suggestions

```
const fetchSuggestions = useCallback(async (input) => {
  if (!LOCATIONIQ_API_KEY || !input.trim()) {
    setSuggestions([]);
    return;
  }
  abortRef.current?.abort();
  const controller = new AbortController();
  abortRef.current = controller;
  ...
```

Before starting a new request, any *previous, still-in-flight* request is cancelled with `.abort()`. This matters because the user is typing fast — without it, an old request for "Ver" could resolve *after* a newer request for "Veritas" and overwrite its results with stale, shorter-query suggestions.

```
const url = `https://api.locationiq.com/v1/autocomplete?key=${LOCATIONIQ_API_KEY}&q=${encodeURIComponent(
const res = await fetch(url, { signal: controller.signal });
```

A plain HTTP GET to LocationIQ's REST API — no SDK. `encodeURIComponent` makes the typed text safe to put in a URL (spaces, commas, etc. get escaped). `limit=5` caps the dropdown at 5 results; `dedupe=1` asks LocationIQ to collapse near-identical results.

Debouncing keystrokes

```
const handleInputChange = (e) => {  
  const text = e.target.value;  
  onChange(text);  
  if (debounceRef.current) clearTimeout(debounceRef.current);  
  debounceRef.current = setTimeout(() => fetchSuggestions(text), 300);  
};
```

Every keystroke updates the visible text immediately (`onChange(text)`), but the network request is delayed by 300ms and *restarted* on every new keystroke (`clearTimeout` cancels the pending one). The network call only actually fires once the user pauses typing for 300ms. This is called **debouncing** — full explanation in §7.

Selecting a suggestion

```
const handleSelect = (suggestion) => {  
  onChange(suggestion.display_name);  
  onSelect?.({  
    address: suggestion.display_name,  
    lat: Number(suggestion.lat),  
    lng: Number(suggestion.lng),  
    placeId: suggestion.place_id,  
  });  
  setSuggestions([]);  
  setOpen(false);  
};
```

LocationIQ returns `lat / lon` as *strings*; `Number(...)` converts them to numbers before handing them up to the parent via `onSelect`. The `?.` (optional chaining) means "call `onSelect` only if the parent actually passed one" — the component still works if a future page uses it without needing coordinates.

Keyboard navigation

```
if (e.key === "ArrowDown") {
  setActiveIndex((i) => (i + 1) % suggestions.length);
} else if (e.key === "ArrowUp") {
  setActiveIndex((i) => (i - 1 + suggestions.length) % suggestions.length);
} else if (e.key === "Enter") {
  if (activeIndex >= 0) handleSelect(suggestions[activeIndex]);
} else if (e.key === "Escape") {
  setOpen(false);
}
```

The `% suggestions.length` (modulo) makes the highlighted row wrap around — pressing Down on the last suggestion jumps back to the first, matching how Google/Bolt-style dropdowns behave.

The `+ suggestions.length` in the Up-arrow case exists purely so JavaScript's modulo of a negative number still lands on a valid, positive index.

What gets rendered

The component returns a positioned wrapper (`relative`) containing the real `<input>` and, only while `open` is true, an absolutely-positioned dropdown (`absolute z-50`) listing each suggestion as a clickable button with a map-pin icon, the place's short name, and its full address in smaller muted text.

7. How the map search was actually built

The original plan: Google Places

Google's Places Autocomplete is the same service behind Bolt/Uber's own address search, so it was the first choice. It requires a Google Cloud project with **billing enabled** — a payment card on file, even though small usage stays within the free monthly credit. That card was rejected repeatedly (a common issue with prepaid/virtual cards or a billing-address mismatch), which blocked that path entirely.

The fallback: LocationIQ

LocationIQ offers the same kind of address-autocomplete data (built on OpenStreetMap), with a free tier (5,000 requests/day) that only needs an email signup — no card. Functionally, it does the same job: send partial text, get back a ranked list of matching places, each with a display name and a coordinate.

WHY NOT JUST QUERY A FULL GEOCODER ON EVERY KEYSTROKE?

Every request has a real cost against a rate limit, and a person typing "23 Adeola Odeku Street" fires roughly 25 events. Two techniques fix that, and both are used here: **debouncing** (wait for a pause in typing before asking the network) and **request cancellation** (drop any in-flight request that's been superseded by a newer keystroke).

The request/response shape

A request looks like:

```
GET https://api.locationiq.com/v1/autocomplete?key=***&q=Veritas&limit=5&dedupe=1&format=json
```

And a single result in the response array looks like:

```
{
  "place_id": "322888729666",
  "lat": "60.4426977",
  "lon": "22.29185533",
  "display_name": "Veritas Stadion, 6, Hippoksentie, ..., Finland",
  "display_place": "Veritas Stadion",
  "display_address": "6, Hippoksentie, ..., Finland"
}
```

`display_place` is the short, bolded line in the dropdown (e.g. a business or street name);

`display_address` is the fuller context shown underneath in muted text; `lat` / `lon` are the "point" — the actual map coordinate captured into `formData` the moment a suggestion is clicked.

End-to-end flow

1. User types into the pickup field → `handleInputChange` updates the visible text instantly.
2. A 300ms silence timer starts (and restarts on every keystroke).
3. Once the pause happens, `fetchSuggestions` calls LocationIQ and stores the results.
4. The dropdown renders the results under the input.
5. User clicks (or arrows-to and presses Enter on) a suggestion → `handleSelect` fires.
6. The booking page's `onSelect` callback stores the final address text *and* `pickup_lat` / `pickup_lng` in `formData`.

8. Input validation

Nigerian phone numbers

```
const NG_PHONE_REGEX = /^(0\d{10}|\+234\d{10})$/;
```

Read as two accepted shapes, either one satisfies the pattern:

- `0\d{10}` — a `0` followed by exactly 10 more digits (11 digits total), e.g. `08031234567`. This is the local format Nigerian numbers are dialled in.
- `\+234\d{10}` — the country code `+234` followed by 10 digits (the same number, minus its leading 0), e.g. `+2348031234567`.

The surrounding `^` and `$` anchor the match to the *whole* string, so `"call 08031234567 now"` would correctly fail — partial matches are rejected, not accepted.

Full names

```
const NAME_REGEX = /^[A-Za-z'-]+(?:\s+[A-Za-z'-]+)+$/;
```

`[A-Za-z'-]+` matches one "word" made only of letters, apostrophes, or hyphens (covering names like `O'Brien` or `Chukwuemeka-Okafor`). The `(?:\s+[A-Za-z'-]+)+` part then requires *at least one more* such word separated by whitespace — in other words, the field must contain a first *and* last name, not a single word, and must not contain digits or stray symbols.

Both regexes back a small helper (`isValidNgPhone` , `isValidFullName`) used in two places: to show a red hint under the field while it's invalid, and to keep the "Next Step" button disabled until it passes.

9. Environment variables

Secrets and configuration live in `.env`, a file that is never committed to source control. This project's build tool (Vite) treats two kinds of variables very differently:

PREFIX	WHERE IT'S READABLE	EXAMPLE HERE
No prefix	Server only (Node process)	<code>DATABASE_URL</code> , <code>ANYTHING_PROJECT_TOKEN</code>
<code>NEXT_PUBLIC_</code>	Server <i>and</i> the browser bundle	<code>NEXT_PUBLIC_LOCATIONIQ_API_KEY</code>

The LocationIQ key *has* to be readable in the browser, because the autocomplete request is fired directly from the user's browser tab, not from this app's own server. That's why it's prefixed `NEXT_PUBLIC_` — anything without that prefix would come back as `undefined` on the client side by design (see `plugins/nextPublicProcessEnv.ts`), which is a deliberate safety rail so a database password can never accidentally leak into client-side JavaScript.

ISN'T A KEY VISIBLE IN THE BROWSER A SECURITY RISK?

Yes, by design — anyone can view it in the Network tab. The mitigation isn't secrecy, it's **restriction**: the key is scoped to only work from approved domains (a "referrer restriction" set in the LocationIQ/Google dashboard) and only has permission to call the autocomplete endpoint, nothing else.

10. What .ts / .tsx / .jsx / .js mean

EXTENSION	CONTAINS	USED FOR, IN THIS PROJECT
<code>.js</code>	Plain JavaScript	Small utility/config files (<code>auth.js</code> , <code>sql.js</code>).
<code>.jsx</code>	JavaScript + JSX (the HTML-like syntax inside <code>return (...)</code>)	Every page and component that renders UI, e.g. <code>booking/page.jsx</code> .
<code>.ts</code>	TypeScript — JavaScript with optional type annotations	Framework plumbing that has no visual output: <code>routes.ts</code> , <code>vite.config.ts</code> .
<code>.tsx</code>	TypeScript + JSX together	<code>root.tsx</code> — the outer HTML shell, which both renders markup <i>and</i> is written with type-checked TypeScript.

TypeScript is a superset of JavaScript: every valid JavaScript file is also valid TypeScript, but TypeScript additionally lets you declare the *shape* of a value — e.g. `type Tree = { path: string; children: Tree[] }` in `routes.ts` — so a tool can catch a whole class of bugs (passing a number where a string was expected, misspelling a property name) before the code ever runs, purely by reading it. None of that annotation exists at runtime: a build step strips it out entirely, which is why `.ts` and `.js` ultimately execute identically. This project mixes both because the type-checking is most valuable in the parts that are easy to get subtly wrong (build configuration, the route-tree walker) and adds the least value in page components that are mostly markup.

11. Running the project locally

1. Install dependencies: `npm install --legacy-peer-deps` (the `--legacy-peer-deps` flag is needed because this project was built against Bun, which resolves peer dependencies more permissively than npm).
2. Add your keys to `apps/web/.env`:

```
NEXT_PUBLIC_LOCATIONIQ_API_KEY=your_key_here
```

3. Start the dev server: `npm run dev` — it prints the local URL (e.g. `http://localhost:4000`).
4. Whenever `.env` changes, the dev server exits itself (a watcher plugin does this on purpose) and must be started again manually.

12. Limitations & next steps

- **Coordinates aren't persisted yet.** `pickup_lat` / `pickup_lng` / `delivery_lat` / `delivery_lng` are captured in the form and sent to the API, but the `deliveries` database table and the `POST` handler don't have matching columns yet, so the server currently ignores them. Adding those columns would be a small, additive database migration.
- **Distance is hard-coded.** `calculateCost()` always assumes a flat 10 km trip. With both coordinates now available, a real distance (haversine formula, or a routing API call) could replace that constant.
- **No automated tests** cover the booking form or the autocomplete component yet.
- **LocationIQ's free tier** is rate-limited (5,000 requests/day); a production launch at scale would need a paid tier or a fallback provider.

End of technical documentation · see the companion Defense Q&A document for likely panel questions